

MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation

The Code Repository: **MiniRAG: Towards Extremely Simple Retrieval-Augmented Generation**

[arXiv](#)

TLDR

MiniRAG is an extremely simple retrieval-augmented generation framework that enables small models to achieve good RAG performance through heterogeneous graph indexing and lightweight topology-enhanced retrieval.

Abstract

The growing demand for efficient and lightweight Retrieval-Augmented Generation (RAG) systems has highlighted significant challenges when deploying Small Language Models (SLMs) in existing RAG frameworks. Current approaches face severe performance degradation due to SLMs' limited semantic understanding and text processing capabilities, creating barriers for widespread adoption in resource-constrained scenarios. To address these fundamental limitations, we present **MiniRAG**, a novel RAG system designed for extreme simplicity and efficiency. **MiniRAG** introduces two key technical innovations: (1) a semantic-aware heterogeneous graph indexing mechanism that combines text chunks and named entities in a unified structure, reducing reliance on complex semantic understanding, and (2) a lightweight topology-enhanced retrieval approach that leverages graph structures for efficient knowledge discovery without requiring advanced language capabilities. Our extensive experiments demonstrate that **MiniRAG** achieves comparable performance to LLM-based methods even when using SLMs while requiring only 25% of the storage space. Additionally, we contribute a comprehensive benchmark dataset LiHua-World for evaluating lightweight RAG systems under realistic on-device scenarios with complex queries.

MiniRAG Framework

MiniRAG employs a streamlined workflow built on the key components: heterogeneous graph indexing and lightweight graph-based knowledge retrieval. This architecture addresses the unique challenges faced by on-device RAG systems, optimizing for both efficiency and effectiveness.

Install

- Install from source (Recommend)

```
cd MiniRAG
pip install -e .
```

- Install from PyPI (Our code is based on [LightRAG](#), so you can install it directly)

```
pip install lightrag-hku
```

Vertex AI & Google Cloud Setup

To enable the LLM Interface module integration via Google Cloud Vertex AI, the system requires installing the official Google SDK package and configuring the cloud access authentication.

1. Install Dependencies for Vertex AI

Run the following command to install the latest Google Gen AI SDK along with error management, retry policies, and data structuring libraries:

```
pip install google-genai tenacity pydantic
```

2. Set Up Application Default Credentials (ADC)

The system utilizes Google Cloud's internal security mechanics via an automated authentication library. Developers can configure the environment using one of the following two options:

Option 1: Using the Google Cloud CLI (Recommended for Local Development)

1. Download and install the [Google Cloud CLI](#) compatible with your operating system.
2. Open your terminal and initialize the GCP account link:

```
gcloud init
```

3. Run the command to generate a local authentication token to grant default application permissions (ADC):

```
gcloud auth application-default login
```

The system will automatically open a browser window for you to log into your Google Cloud account. Once completed, the security credential `.json` file will be stored locally on your machine, and the API calls will automatically detect it.

Option 2: Using a Service Account Key (Recommended for Production / Docker / Remote Servers)

1. Navigate to the Google Cloud Console, create a **Service Account**, and assign at least the `Vertex AI User` role.
2. Create and download a private key in `.json` file format.
3. Declare the file path using system environment variables:

- **Linux / macOS:**

```
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your/key-file.json"
```

- **Windows (PowerShell):**

```
$env:GOOGLE_APPLICATION_CREDENTIALS="C:\path\to\your\key-file.json"
```

- **Windows (Command Prompt - CMD):**

```
set GOOGLE_APPLICATION_CREDENTIALS="C:\path\to\your\key-file.json"
```

OpenBookQA Quick Start

This repository is currently set up to run MiniRAG on OpenBookQA using the helper scripts in the project root.

1. Prepare the OpenBookQA dataset

Install the dataset dependency once:

```
pip install datasets
```

Then build the local MiniRAG-ready dataset:

```
python prepare_openbookqa.py
```

This creates:

- `./dataset/OpenBookQA/data/` : one science fact per `.txt` file
- `./dataset/OpenBookQA/qa/query_set.csv` : multiple-choice test questions in the format expected by `Step_1_QA.py`

If you have already prepared the dataset, you can skip this step.

2. Build the OpenBookQA index

Run indexing once:

```
python ./reproduce/Step_0_index.py \  
--model GLM \  
--workingdir ./OpenBookQA-index \  
--datapath ./dataset/OpenBookQA/data/
```

This creates the graph/vector index in `./OpenBookQA-index`.

3. Run MiniRAG on OpenBookQA

The default OpenBookQA MiniRAG helper is:

```
bash run_openbookqa.sh
```

It runs:

```
python ./reproduce/Step_1_QA.py \
  --model GLM \
  --workingdir ./OpenBookQA-index \
  --datapath ./dataset/OpenBookQA/data/ \
  --querypath ./dataset/OpenBookQA/qa/query_set.csv \
  --outputpath ./logs/OpenBookQA_output.csv \
  --answer-format multiple_choice \
  --overwrite-output
```

Notes:

- `--answer-format multiple_choice` forces the final output into A/B/C/D, which is needed for OpenBookQA evaluation.
- `--overwrite-output` starts a fresh run instead of resuming an existing CSV.
- If your GPU memory is tight, add `--use-fp16` to `Step_1_QA.py`.

4. Run the naive baseline on OpenBookQA

To run the existing naive RAG baseline:

```
bash run_openbookqa_naive.sh
```

It writes predictions to:

- `./logs/OpenBookQA_naive_output.csv`

The underlying command is:

```
python ./reproduce/Step_1_QA.py \
  --model GLM \
  --workingdir ./OpenBookQA-index \
  --datapath ./dataset/OpenBookQA/data/ \
  --querypath ./dataset/OpenBookQA/qa/query_set.csv \
  --outputpath ./logs/OpenBookQA_naive_output.csv \
  --rag-mode naive \
  --prediction-col naive \
  --answer-format multiple_choice \
  --overwrite-output
```

5. Score OpenBookQA results

Score MiniRAG:

```
python score_openbookqa.py \
  --inputpath ./logs/OpenBookQA_output.csv \
  --prediction-col minirag
```

Score naive:

```
python score_openbookqa.py \
  --inputpath ./logs/OpenBookQA_naive_output.csv \
  --prediction-col naive
```

If you want to inspect wrong predictions:

```
python score_openbookqa.py \
  --inputpath ./logs/OpenBookQA_output.csv \
  --prediction-col minirag \
  --show-mistakes
```

There is also a simple wrapper:

```
bash run_score.sh openbookqa minirag
bash run_score.sh openbookqa naive
```

6. Important files

- `prepare_openbookqa.py` : downloads and converts OpenBookQA into MiniRAG's local format
- `reproduce/Step_0_index.py` : builds the OpenBookQA graph/vector index
- `reproduce/Step_1_QA.py` : runs MiniRAG or naive RAG on the OpenBookQA questions
- `run_openbookqa.sh` : MiniRAG OpenBookQA helper
- `run_openbookqa_naive.sh` : naive OpenBookQA helper
- `score_openbookqa.py` : computes multiple-choice accuracy from the output CSV
- `run_score.sh` : simple wrapper for scoring

Reproduce

The main OpenBookQA workflow in this repository is:

1. `python prepare_openbookqa.py`
2. `python ./reproduce/Step_0_index.py --model GLM --workingdir ./OpenBookQA-index --datapath ./dataset/OpenBookQA/data/`
3. `bash run_openbookqa.sh`
4. `python score_openbookqa.py --inputpath ./logs/OpenBookQA_output.csv --prediction-col minirag`

Dataset: OpenBookQA

This repository includes a helper script for converting OpenBookQA into MiniRAG's local file-based format.

- Source dataset: `allenai/openbookqa`
- Converter: `prepare_openbookqa.py`
- Output facts: `./dataset/OpenBookQA/data/fact_XXXX.txt`
- Output evaluation set: `./dataset/OpenBookQA/qa/query_set.csv`

The generated `query_set.csv` keeps the original multiple-choice question text and stores the gold answer as answer text. During evaluation, `Step_1_QA.py` can be switched to `--answer-format multiple_choice` so MiniRAG returns `A/B/C/D` for direct scoring with `score_openbookqa.py`.

Thank you for your interest in our work!